

Microservicios en C++

Antonio Espín Herranz

Contenidos

- HTTP y REST
- gRPC y WebSockets
- Instalación de Crow mediante vcpkg.
- Estructura de un microservicio en C++.
- Rutas, controladores y manejo de peticiones GET/POST/PUT/DELETE.
- Gestión de JSON: integración con nlohmann/json.
- Manejo de errores y buenas prácticas en respuesta HTTP.
- Logging básico del servicio.

HTTP y REST

Introducción Protocolos HTTP, HTTPs y WebSockets

HTTP

- Es el protocolo base de la web. Permite la comunicación entre clientes (como navegadores) y servidores mediante un modelo solicitud-respuesta.
- **Características:**
 - **Unidireccional:** El cliente envía una solicitud, el servidor responde.
 - **Sin estado:** No guarda información entre solicitudes (aunque puede usarse con cookies o sesiones).
 - Usa el **puerto 80** por defecto.
 - **Formato textual:** Las solicitudes y respuestas son legibles y estructuradas.

HTTPs

- **Http secure**

- Es la versión segura de HTTP. Utiliza TLS/SSL para cifrar la comunicación entre cliente y servidor.

- **Características:**

- **Cifrado de extremo a extremo:** Protege datos sensibles como contraseñas o tarjetas.
- **Autenticación:** Verifica que estás hablando con el servidor correcto mediante certificados digitales.
- **Integridad:** Evita que los datos sean modificados durante la transmisión.
- Usa el **puerto 443** por defecto.

WebSockets

- Es un protocolo de comunicación **bidireccional y persistente** que permite que cliente y servidor intercambien datos en tiempo real sin necesidad de múltiples solicitudes HTTP
- **Características:**
 - **Full-duplex:** Ambos lados pueden enviar y recibir datos simultáneamente.
 - **Conexión persistente:** Se mantiene abierta, ideal para apps en tiempo real.
 - **Menor latencia:** Los datos se envían tan pronto como están disponibles.
 - Usa el **puerto 80 (ws://)** o **443 (wss://)** dependiendo de si es seguro.

WebSockets

- **Casos de uso:**
 - Chats en vivo
 - Juegos multijugador
 - Notificaciones en tiempo real
 - Streaming de datos

Comparativa

Protocolo	Dirección	Persistencia	Seguridad	Ideal para...
HTTP	Cliente → Servidor	No	No	Páginas web estáticas
HTTPS	Cliente → Servidor	No	Sí	Formularios, login, e-commerce
WebSockets	Bidireccional	Sí	Opcional	Apps en tiempo real

Comparativa

- **Diferencias entre HTTP/1.1 y HTTP/2**

Característica	HTTP/1.1	HTTP/2
Multiplexación	✗ No (una petición por conexión)	✓ Sí (varias peticiones simultáneas)
Compresión de headers	✗ No	✓ Sí (HPACK)
Prioridad de streams	✗ No	✓ Sí
Formato de mensajes	Texto plano	Binario
Rendimiento	Menor	Mayor (menos latencia, más eficiencia)

Comparativa entre APIs C++

- HTTP/1.1 | HTTP/2 | WebSocket

API/Librería	HTTP/1.1	HTTP/2	¿Se puede elegir?	WebSocket
Crow	✓	🏠 Parcial (experimental)	No (solo HTTP/1.1 estable)	✓ Sí LibHunt
Boost.Beast	✓	✗ No nativo	No (solo HTTP/1.1)	✓ Sí Github +1
gRPC (C++)	✗	✓ Requiere HTTP/2	No (solo HTTP/2)	✗ No (usa streaming propio)
Restbed	✓	✗ No	No	✗ No
cpp-httpplib	✓	✓ (experimental con ngtcp2)	Sí (por configuración)	✗ No
nghttp2	✗	✓ Solo HTTP/2	No (solo HTTP/2)	✗ No
Pistache	✓	✗ No	No	✗ No

WebSockets

- **Crow** y **Boost.Beast** soportan WebSockets
- **gRPC** no usa WebSockets, ofrece streaming bidireccional sobre HTTP/2

Peticiones o verbos HTTP

Verbo	Acción principal	Equivalente CRUD	Uso típico en APIs REST
GET	Obtener datos	Read	Consultar información (ej. productos)
POST	Crear nuevos datos	Create	Registrar usuario, añadir producto
PUT	Actualizar datos existentes	Update	Modificar perfil, cambiar contraseña
DELETE	Eliminar datos	Delete	Borrar cuenta, eliminar comentario

Ejemplos

- **GET** /libros → devuelve la lista de libros.
- **GET** /libros/123 → devuelve el libro con ID 123.
- **POST** /libros → crea un nuevo libro (con datos en el cuerpo).
- **PUT** /libros/123 → actualiza el libro con ID 123.
- **DELETE** /libros/123 → elimina el libro con ID 123.

Comparativa entre REST y gRPC

Característica	REST (HTTP/JSON)	gRPC (HTTP/2 + Protobuf)
 Protocolo	HTTP/1.1	HTTP/2
 Formato de datos	JSON	Protocol Buffers (binario)
 Definición de API	Manual (OpenAPI/Swagger opcional)	Automática con archivos <code>.proto</code>
 Comunicación	Sincrónica (por defecto)	Sincrónica y asíncrona (streaming bidireccional)
 Validación	Manual	Validación automática por esquema <code>.proto</code>
 Compatibilidad	Universal (navegadores, herramientas web)	Limitada en navegadores, ideal para backend
 Rendimiento	Medio	Alto (más rápido y eficiente)
 Multilenguaje	Muy buena	Excelente (con generación automática)
 Seguridad	TLS, OAuth, JWT	TLS, autenticación personalizada

REST

- **Ideal para:**

- APIs públicas o abiertas.
- Aplicaciones web y móviles.
- Sistemas donde la compatibilidad con navegadores es clave.

- **Evítalo si:**

- Necesitas rendimiento extremo o streaming bidireccional.
- Quieres evitar la sobrecarga de JSON en servicios internos

gRPC

- **Ideal para:**

- Comunicación entre microservicios backend.
- Sistemas embebidos, telecomunicaciones, alto rendimiento.
- Streaming de datos en tiempo real (IoT, juegos, ML).

- **Evítalo si:**

- Tu cliente es un navegador (gRPC no funciona directamente en ellos).
- No quieres depender de herramientas como protoc para generar código.

Uso por sectores

Sector	REST	gRPC
Web pública	Ok	Limitado en navegadores
Microservicios	Ok	Más eficiente
Sistemas Embebidos	Pesado	Ligero y rápido
Telecomunicaciones	No	Streaming, binario, eficiente
IA / ML	Para Dashboards	Para inferencia distribuida

Implementación

- **gRPC no usa directamente los verbos HTTP como POST, GET, PUT o DELETE.**
- **Se puede modelar esas operaciones típicas de un microservicio** en C++ usando gRPC, pero con un enfoque diferente.
 - Hay que diseñar un fichero **.proto** que sea equivalente a REST

Ejemplo .proto

```
service ProductoService {  
    rpc CrearProducto (Producto) returns (Respuesta);    // POST  
    rpc ObtenerProducto (ProductoID) returns (Producto); // GET  
    rpc ActualizarProducto (Producto) returns (Respuesta); // PUT  
    rpc EliminarProducto (ProductoID) returns (Respuesta); // DELETE  
}
```

Peticiones

REST (HTTP)	gRPC (RPC)
GET /producto/123	ObtenerProducto(ProductID)
POST /producto	CrearProducto(Producto)
PUT /producto/123	ActualizarProducto(Producto)
DELETE /producto/123	EliminarProducto(ProductID)

Compilar fichero .proto

- Al compilar se generan clases C++ con métodos virtuales para su implementación:

```
class ProductoServiceImpl final : public ProductoService::Service {  
    grpc::Status CrearProducto(grpc::ServerContext* context,  
                               const Producto* request,  
                               Respuesta* response) override {  
        // lógica de creación  
        return grpc::Status::OK;  
    }  
};
```

Ventajas

- Tipado fuerte y validación automática.
- Comunicación binaria, más rápida que JSON.
- Streaming bidireccional si lo necesitas.
- Generación automática de cliente y servidor.

Peticiones Http

Servidor HTTP

boost.beast

Ejemplo: Servidor Http

- Implementar un servidor Http: Tipos utilizados y pasos para implementar el Servidor.
- Objetos relacionados:
 - **boost::asio::io_context**
 - El sistema de eventos.
 - Coordina todas las operaciones asincrónicas.
 - **boost::asio::ip::tcp::acceptor**
 - Escucha conexiones entrantes en un puerto específico.
 - **boost::asio::ip::tcp::socket**
 - Representa una conexión TCP con un cliente.
 - **boost::beast::http::request y http::response**
 - Manejan las peticiones y respuestas HTTP.
 - **boost::beast::tcp_stream**
 - Encapsula el socket y proporciona operaciones de lectura/escritura con Beast.
 - **Buffers (boost::beast::flat_buffer)**
 - Para almacenar temporalmente los datos recibidos.
 - **Una clase Session**
 - Para manejar cada conexión de forma independiente.
 - Definir cómo se lee la petición y se envía la respuesta.

Servidor Http - Tipos

- **boost::asio::io_context**
 - Es el motor de eventos de Boost.Asio.
 - Gestiona operaciones de entrada/salida (I/O), como aceptar conexiones o leer datos.
 - Se usa para crear el **acceptor** y los **sockets**.

Servidor Http - Tipos

- **boost::asio::ip::tcp::acceptor**

- Se encarga de escuchar en un puerto TCP.
- Espera conexiones entrantes y las acepta, creando un socket para cada cliente.

- **boost::asio::ip::tcp::socket**

- Representa una conexión TCP con un cliente.
- Se usa para leer la petición HTTP y enviar la respuesta.

Servidor Http - Tipos

- **boost::beast::flat_buffer**
 - Buffer de Beast para almacenar datos leídos del socket.
 - Necesario para las operaciones de lectura HTTP.
- **boost::beast::http::request<T> y http::response<T>**
 - Representan mensajes HTTP.
 - El tipo T indica el tipo de cuerpo (para un mensaje, string_body para texto plano).
 - Se usan para leer la petición del cliente y construir la respuesta.

Servidor Http - Tipos

- **boost::beast::http::read y http::write**
 - Funciones para leer una petición HTTP desde el socket y escribir una respuesta.
 - Son operaciones sincrónicas, pero también existen versiones asincrónicas (**async_read**, **async_write**).

Pasos para implementar el Servidor Http I

- **1 - Inicialización**

- `HttpServer server(ioc, 8080);`

- **2 - Escucha de conexiones:**

- `tcp::acceptor acceptor(ioc, tcp::endpoint(tcp::v4(), port));`
- El acceptor se configura para escuchar en IPv4 y en el puerto indicado

- **3 - Bucle de atención:**

- `for (;;) {`
- `tcp::socket socket(ioc_);`
- `acceptor_.accept(socket);`
- `handle_request(socket);`
- `}`

El servidor entra en un bucle infinito:

Acepta una conexión entrante.

Crea un socket para comunicarse con el cliente.

Llama a `handle_request()` para procesar la petición.

Pasos para implementar el Servidor Http II

- **4 – Procesamiento de la petición:**

- `http::request<http::string_body> req;`
- `http::read(socket, buffer, req);`
- Se lee la petición HTTP del cliente.
- Se almacena en el objeto req.

Pasos para implementar el Servidor Http III

- **5 – Construcción de la respuesta:**

- `http::response<http::string_body> res{http::status::ok, req.version()};`
- `res.set(http::field::server, "Beast/1.0");`
- `res.set(http::field::content_type, "text/plain");`
- `res.body() = "Mensaje desde el Servidor";`
- `res.prepare_payload();`
- *Se crea una respuesta con estado 200 OK.*
- *Se añaden encabezados como Server y Content-Type.*
- *Se asigna el cuerpo del mensaje.*
- *prepare_payload() calcula automáticamente el tamaño del cuerpo.*

Pasos para implementar el Servidor Http IV

- `http::write(socket, res);`
- Se envía la respuesta al cliente a través del socket.

gRPC / WebSockets

gRPC

Los proyectos con C++ 17

gRPC

- **Google Remote Procedure Call**
- Es un framework de comunicación de alto rendimiento y código abierto que permite a aplicaciones intercambiar datos entre sí de forma eficiente, rápida y estructurada.
- Fue desarrollado por Google y se basa en el protocolo **HTTP/2** y en **Protocol Buffers (protobuf)** para la serialización de datos.

Introducción

- **Streaming bidireccional**

- gRPC permite:
- **Streaming del servidor:** el servidor envía múltiples respuestas.
- **Streaming del cliente:** el cliente envía múltiples peticiones.
- **Streaming bidireccional:** ambos envían y reciben datos en tiempo real.
- Ideal para chats, dashboards en vivo, sensores, etc.

- **Interoperabilidad entre lenguajes**

- Puedes tener:
- Un servidor en C++
- Un cliente en Python, Go, JavaScript, etc.
- Todo gracias a que comparten el mismo **.proto**.

gRPC

- Permite definir **servicios** y sus métodos usando archivos **.proto**.
- Un archivo **proto** es un **IDL** (lenguaje de definición de interface) y después se puede **generar código** en C++
- Genera automáticamente el código cliente y servidor en múltiples lenguajes (C++, Go, Java, Python, etc.).
- Usa **llamadas a procedimientos remotos (RPC)** para que una aplicación pueda ejecutar funciones en otra como si fueran locales.

gRPC: Ventajas

- **Comunicación eficiente**
 - Usa **HTTP/2**, lo que permite multiplexación de conexiones, compresión de cabeceras y streaming bidireccional.
- **Serialización rápida**
 - Utiliza **Protocol Buffers**, que son más compactos y rápidos que JSON o XML.
- **Multilenguaje**
 - Compatible con muchos lenguajes: ideal para arquitecturas de microservicios heterogéneas.
- **Seguridad**
 - Soporta **TLS** para cifrado de extremo a extremo.

Casos de uso típicos

Sector	Ejemplo de uso con gRPC
Microservicios	Comunicación entre servicios backend
Telecomunicaciones	Transmisión de datos entre nodos de red
IoT / sistemas embebidos	Comunicación eficiente entre dispositivos
Juegos online	Sincronización de estado entre cliente y servidor
Machine Learning	Servir modelos de IA con baja latencia

Componentes necesarios

- **protoc** es un **compilador** que tomará el archivo **proto** como entrada y generará las estructuras de mensajes y las interfaces de servicio en el lenguaje deseado, en nuestro caso, C++.
- **El archivo .proto** es un archivo **IDL** donde definimos las estructuras de paso de mensajes y las API de rpc de acuerdo con las especificaciones de **Protobuf**.
- **Complemento gRPC** para el lenguaje específico:
 - Existen complementos gRPC separados para diferentes lenguajes de programación y el protocolo los requerirá para generar los archivos fuente según las interfaces de servicio definidas en el IDL.

Instalación

- Se instala con el gestor de paquetes: **vcpkg**
 - **vcpkg install grpc**
 - **vcpkg integrate install**
- Visual Studio debería de reconocer:
#include <grpcpp/grpcpp.h>
#include <grpcpp/server.h>
#include <grpcpp/server_builder.h>
- Para la compilación de archivos **.proto**
- Necesitamos instalar:
 - **vcpkg install protobuf**
 - **vcpkg integrate install**

Instalación

- Dentro de la instalación de **vcpkg** tenemos que localizar el comando **protoc** y añadirlo al **PATH**.
- Por ejemplo:
 - C:\vcpkg\installed\x64-windows\tools\protobuf**protoc.exe**
- Hay un plugin **grpc_cpp_plugin.exe** que se utiliza para generar código C++, y debería de estar también en:
 - C:\vcpkg\installed\x64-windows\tools\grpc**grpc_cpp_plugin.exe**
 - Si no se instaló correctamente lanzar estos comandos:
 - **vcpkg remove grpc**
 - **vcpkg install grpc --recurse**
 - **vcpkg integrate install**

Pasos para desarrollar con gRPC cliente / servidor para enviar un mensaje

- **1. Definir el servicio con Protocols Buffers**
- Crea un archivo .proto que describa:
 - El servicio (por ejemplo, SaludoService)
 - Los métodos (por ejemplo, DiHola)
 - Los mensajes de entrada y salida (por ejemplo, SolicitudSaludo, RespuestaSaludo)
- Se puede ubicar en una carpeta de la solución aparte.

Pasos para desarrollar con gRPC cliente / servidor para enviar un mensaje

- **2. Generar el código fuente desde el .proto**
- Usa protoc junto con el plugin de gRPC para generar:
 - Archivos .pb.h y .pb.cc (mensajes)
 - Archivos .grpc.pb.h y .grpc.pb.cc (servicio gRPC)
- Esto se hace en la línea de comandos que invoca el compilador de protobuf.

Pasos para desarrollar con gRPC cliente / servidor para enviar un mensaje

- **3. Implementar el servidor**
- Crea un archivo fuente para el servidor.
- Incluye:
 - La implementación de la clase del servicio (hereda de la interfaz generada)
 - La lógica del método (por ejemplo, devolver “Hola, Juan”)
 - La configuración del servidor (puerto, credenciales, etc.)

Pasos para desarrollar con gRPC cliente / servidor para enviar un mensaje

- **4. Implementar el cliente**
- Crea un archivo fuente para el cliente.
- Incluye:
 - La creación del stub (cliente gRPC)
 - La construcción del mensaje de solicitud
 - La llamada al método remoto
 - La recepción y manejo de la respuesta

Pasos para desarrollar con gRPC cliente / servidor para enviar un mensaje

- **5. Compilar el proyecto**
- Usa CMake o Visual Studio para compilar:
 - Los archivos generados por protoc
 - El servidor y el cliente
 - Las dependencias de gRPC y protobuf
- **6. Ejecutar**
- Primero ejecuta el **servidor** para que escuche en el puerto.
- Luego ejecuta el **cliente** para enviar la solicitud y recibir la respuesta.

¿Cómo se usa?

- El servicio se define en un archivo **.proto**

```
service Saludo {  
  rpc DiHola (Mensaje) returns (Respuesta);  
}
```
- Se compila con **protoc**
 - Implementar el servidor y el cliente en el lenguaje elegido.
- gRPC genera código a partir de contratos.
- El archivo **.proto** define la estructura y servicios y **protoc genera la parte de comunicación.**
- La lógica de que hace el cliente y el servidor la escribimos nosotros.

Estructura básica del archivo: **saludo.proto**

```
syntax = "proto3";           // Versión del lenguaje de Protobuf  
package saludo;             // Nombre del paquete (opcional pero recomendable)
```

```
service Saludo {             // Definición del servicio gRPC  
  rpc DiHola (Solicitud) returns (Respuesta); // Método RPC  
}
```

```
message Solicitud {          // Mensaje de entrada  
  string nombre = 1;  
}
```

```
message Respuesta {         // Mensaje de salida  
  string mensaje = 1;  
}
```

Generar código

protoc

--cpp_out=.

--grpc_out=.

--plugin=protoc-gen-grpc=D:\vcpkg\installed\x64-windows\tools\grpc\grpc_cpp_plugin.exe

saludo.proto

- El comando genera:
 - **saludo.pb.h** y **saludo.pb.cc**: clases para los mensajes (Solicitud, Respuesta)
 - **saludo.grpc.pb.h** y **saludo.grpc.pb.cc**: clases para el servicio (**Saludo::Service**, **Saludo::Stub**)

Escribir el servidor

- Crea una clase que herede de **Saludo::Service**.
- Implementa la lógica del método.
- Usa `grpc::ServerBuilder` para iniciar el servidor.

Escribir el cliente

- Usa **Saludo::Stub** para conectarte al servidor.
- Llama a **DiHola()** pasando una instancia de **Solicitud**.
- Recibe la respuesta en una instancia de **Respuesta**.

En Visual Studio

- Crear una solución:
 - En la carpeta Proto mantener el fichero .proto y los ficheros generados, se llamarán:
 - **saludo.pb.h**
 - **saludo.pb.cc**
 - **saludo.grpc.pb.h**
 - **saludo.grpc.pb.cc**
 - Crear dos proyectos en la solución:
 - **Servidor y Cliente**
 - En cada proyecto hay que vincular los ficheros generados con **protoc**
 - Se pueden agregar como elementos existentes con botón derecho sobre cada proyecto.
 - Después en propiedades de cada proyecto, ir a C/C++ → General → Directorios de inclusión adicionales y añadir **\$(SolutionDir)Proto**

saludo.pb.h

- **Encabezado de Protocol Buffers**
- Define las clases C++ para los mensajes que has declarado en el .proto, como:
 - **Solicitud**
 - **Respuesta**
- Incluye métodos para:
 - Acceder y modificar campos (set_nombre(), nombre(), etc.)
 - Serializar y deserializar los mensajes
 - Comparar, copiar, limpiar, etc.

saludo.pb.cc

- **Implementación de Protocol Buffers**
- Contiene el código fuente que implementa las clases declaradas en **saludo.pb.h**
- Incluye la lógica de serialización, construcción de objetos, y manejo interno de campos

saludo.grpc.h

- **Encabezado de gRPC**
- Define las interfaces para el servicio gRPC que declaraste en el .proto, por ejemplo:
 - Clase abstracta Saludo::**Service** con métodos virtuales como DiHola(...)
 - Clase Saludo::**Stub** para que el cliente invoque el servicio
- También incluye definiciones para el contexto de llamada (grpc::ServerContext, grpc::ClientContext)

saludo.grpc.pb.cc

- **Implementación de gRPC**
- Implementa el **stub** del cliente y el registro del servicio en el servidor
- Contiene el código que conecta los métodos RPC con el transporte gRPC
- Maneja la serialización de mensajes entre cliente y servidor usando Protocol Buffers

- **Tipos de datos más ricos**

- int32, int64, float, double, bool, bytes
- enum para valores constantes y repeated para listas

```
message Usuario {  
    string nombre = 1;  
    int32 edad = 2;  
    repeated string intereses = 3;  
}
```

- **Streaming de datos (gRPC permite streaming bidireccional)**

```
service Chat {  
    rpc Conversacion(stream Mensaje) returns (stream Mensaje);  
}
```

- **Mensajes anidados y reutilizables:**

```
message Direccion {  
    string calle = 1;  
    string ciudad = 2;  
}
```

```
message Perfil {  
    string nombre = 1;  
    Direccion direccion = 2;  
}
```

IDL

- Enums para estados o tipos:

```
enum Estado {  
    ACTIVO = 0;  
    INACTIVO = 1;  
    SUSPENDIDO = 2;  
}
```

- Opciones personalizadas y extensiones

- Opciones para definir metadatos, como validaciones o documentación extra.

```
message Producto {  
    string nombre = 1 [(validate.rules).string.min_len = 3];  
}
```

IDL

- **Los números de los campos son identificadores únicos (en formato binario)**
- **Permite que sean compactos y eficientes:**
 - **Solo se transmiten los números**, no los nombres. Esto:
 - Reduce el tamaño del mensaje.
 - Permite renombrar campos sin romper compatibilidad.
 - Hace que el protocolo sea más rápido y eficiente.
- ***Es crítico para la compatibilidad, cambiar el número rompe la compatibilidad con versiones anteriores.***

IDL

- Los números deben estar entre 1 y $2^{29} - 1$ (excepto los reservados).
- No se deben reutilizar ni cambiar una vez en uso.
- Se pueden reservar números: reserved 3,4,5;
- Los números se reservan dentro del mensaje.
 - Se suelen reservar cuando se elimina un campo y su número correspondiente, en este caso es mejor reservarlo para no volver a utilizarlo.

Aplicaciones comunes entre gRPC / C++

- **Sistemas distribuidos y microservicios**

- Comunicación entre servicios en arquitecturas modernas.
- Sustituto de REST cuando se necesita eficiencia.
- Ejemplo: backend de una plataforma de streaming o e-commerce.

- **Redes y telecomunicaciones**

- Transmisión de datos en tiempo real.
- Control de dispositivos de red, routers, switches.
- Ejemplo: gestión de infraestructura 5G o SDN (Software Defined Networking).

Aplicaciones comunes entre gRPC / C++

- **IoT y dispositivos embebidos**

- Comunicación entre sensores, gateways y servidores.
- gRPC es ligero y eficiente, ideal para dispositivos con recursos limitados.

- **Gaming y simulaciones**

- Sincronización de estado entre cliente y servidor.
- Envío de eventos en tiempo real (movimiento, colisiones, etc.).
- Ejemplo: juegos multijugador con servidores dedicados.

- **Machine Learning y procesamiento de datos**

- Comunicación entre modelos, servidores de inferencia y clientes.
- Ejemplo: un cliente C++ que envía imágenes a un servidor Python con TensorFlow.

Aplicaciones comunes entre gRPC / C++

- **Finanzas y trading algorítmico**

- Envío de datos de mercado en tiempo real.
- Ejecución de órdenes con baja latencia.
- Ejemplo: plataformas de trading de alta frecuencia.

- **Automoción y robótica**

- Comunicación entre módulos de control (sensores, actuadores, navegación).
- Ejemplo: vehículos autónomos que intercambian datos entre subsistemas.

gRPC vs REST

Característica	gRPC	REST
Protocolo	HTTP/2	HTTP/1.1
Serialización	Binaria (Protobuf)	Texto (JSON)
Rendimiento	Alto	Medio
Streaming	Bidireccional	Limitado
Tipado	Fuerte (IDL)	Débil
Soporte multilenguaje	Excelente	Bueno

Servicio con streaming

```
syntax = "proto3";  
package chat;
```

```
service ChatService {  
  rpc Conversacion(stream Mensaje) returns (stream Mensaje);  
}
```

```
message Mensaje {  
  string usuario = 1;  
  string texto = 2;  
}
```

Servidor simplificado

```
class ChatServiceImpl final : public ChatService::Service {
    Status Conversacion(ServerContext* context,
                        ServerReaderWriter<Mensaje, Mensaje>* stream) override {
        Mensaje mensaje;
        while (stream->Read(&mensaje)) {
            std::cout << "Recibido de " << mensaje.usuario() << ": " << mensaje.texto() << std::endl;

            Mensaje respuesta;
            respuesta.set_usuario("Servidor");
            respuesta.set_texto("Hola " + mensaje.usuario() + ", recibí tu mensaje.");
            stream->Write(respuesta);
        }
        return Status::OK;
    }
};
```

Cliente simplificado

```
std::unique_ptr<ChatService::Stub> stub = ChatService::NewStub(channel);  
ClientContext context;  
std::shared_ptr<ClientReaderWriter<Mensaje, Mensaje>> stream(stub->Conversacion(&context));
```

```
Mensaje mensaje;  
mensaje.set_usuario("Cliente1");  
mensaje.set_texto("¡Hola servidor!");  
stream->Write(mensaje);
```

```
Mensaje respuesta;  
while (stream->Read(&respuesta)) {  
    std::cout << "Respuesta: " << respuesta.texto() << std::endl;  
}
```

Enlaces

- Configurar **gRPC** en **Windows**:
 - <https://sanoj.in/2020/05/07/working-with-grpc-in-windows.html>

WebSockets
boost.beast

WebSockets

- Dentro de la librería **boost.beast** con **WebSockets** podemos implementar:
 - Un **cliente WebSocket**: conectar con un servidor WebSocket remoto (para consumir datos en tiempo real).
 - Un **Servidor WebSocket**: crear un servidor que escuche conexiones WebSocket entrantes y gestionar múltiples clientes simultáneamente.

Realizar el handshake (establecer la conexión)

- Gestiona el **handshake HTTP** inicial que convierte una conexión HTTP en una conexión WebSocket.
- Permite personalizar los encabezados del handshake para añadir autenticación, tokens, etc.

Enviar y recibir mensajes

- Lectura y escritura **síncrona** (read, write)
- Lectura y escritura **asíncrona** (async_read, async_write)
- Compatible con mensajes de **texto** y **binarios**
- Se pueden utilizar **buffers dinámicos** o **estáticos**, y gestionar los mensajes con precisión.

Soporte para WebSocket seguro (WSS)

- Integración con **SSL / TLS** mediante **Boost.asio**
- Establecer conexiones seguras usando certificados y clases privadas
- Ideal para aplicaciones que requieren confidencialidad (como chats, trading, IoT)

Control de flujo y gestión de errores

- Manejo de errores detallado con `boost::system::error_code`
- Control de cierre de conexión (close) con códigos estándar WebSocket
- Detección de desconexiones, timeouts, y errores de protocolo

Personalización avanzada

- Puedes acceder directamente a los **encabezados HTTP** del handshake
- Configurar opciones como:
 - **Fragmentación de mensajes**
 - **Tamaño máximo de buffer**
 - **Control de ping/pong** para mantener viva la conexión

Integración con otras tecnologías

- Compatible con **Boost.asio coroutines** (co_spawn, awaitable)
- Puedes combinar WebSockets con **HTTP/REST**, **TCP**, o **SSL** en una misma aplicación
- Ideal para servidores híbridos que ofrecen tanto APIs como canales WebSocket

Ejemplos de uso

- Chat en tiempo real
- Streaming de datos financieros
- Juegos multijugador
- Comunicación entre dispositivos IoT
- Actualización en vivo de interfaces web.

Servidor WebSocket

- Crea el contexto de E/S (io_context)
- Abre el socket de escucha en el puerto 80 (tcp::acceptor)
- Entra en un bucle infinito:
 - Acepta nuevas conexiones entrantes (acceptor.**accept**)
 - Por cada conexión, crea un nuevo hilo con std::**thread**{do_session, std::move(socket)}.**detach**()
 - Desconectamos el hilo del hilo principal para que no se quede esperando y se ejecuta en paralelo.
- En cada hilo (do_session):
 - Realiza el handshake WebSocket
 - Entra en un bucle de lectura:
 - Recibe mensajes
 - Los devuelve como eco
 - Si el cliente se desconecta, lanza una excepción que se captura

Cliente WebSocket

- Crea el contexto de E/S (io_context)
- Resuelve el host localhost y el puerto 80
- Conecta el socket TCP
- Realiza el handshake WebSocket
- Envía un mensaje (write)
- Espera la respuesta del servidor (read)
- Imprime la respuesta
- Cierra la conexión (close_code::normal)

Testear Servidor WebSocket

- Disponemos de una herramienta online para testear el servidor:
- <https://piehost.com/websocket-tester>
- Y luego nos conectamos a:
- **ws://localhost:8080**

Web Socket Secure

WSS

WSS (WebSocket Secure)

- Necesitamos hacer algunos cambios:
 - Crear un contexto `ssl:context`
 - Cambiar el tipo WebSocket stream
 - Realizar el handshake SSL antes del handshake WebSocket
 - El puerto tiene que ser **443**
-
- Necesitamos la herramienta **openssl**, y generar un certificado

Tipos de certificado

Tipo de certificado	¿Quién lo emite?	¿Dónde se usa?	¿Requiere hardware?
DNI electrónico (DNle)	Gobierno (Policía Nacional en España)	Trámites oficiales, firma electrónica	Sí, lector de tarjetas
Certificado FNMT	Fábrica Nacional de Moneda y Timbre	Administración pública, firma digital	No
Certificado local (OpenSSL)	Tú mismo con OpenSSL	Desarrollo, pruebas, servidores propios	No
Certificados SSL/TLS	Autoridades de certificación (CA)	Sitios web seguros (HTTPS)	No
Certificados de firma de código	CA como DigiCert, Sectigo	Firmar software, garantizar integridad	No

CA: Autoridad de certificación

openssl

- Comando: ***openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365***
- El comando:
 - Crea un certificado autofirmado (no emitido por una CA).
 - Clave privada RSA de 2048 bits
 - Pide una serie de datos
- Así como una contraseña para encriptar la clave:
 - Se genera como resultado dos archivos:
 - **key.pem** → clave privada
 - **cert.pem** → tu certificado público autofirmado
 - **Estos dos ficheros son necesarios para WSS**

Opciones

Opción	Descripción
<code>openssl</code>	Llama al programa OpenSSL, una herramienta de línea de comandos para criptografía.
<code>req</code>	Indica que se va a generar una solicitud de certificado (CSR) o un certificado X.509.
<code>-x509</code>	En lugar de generar una CSR, se genera directamente un certificado X.509 autofirmado.
<code>-newkey rsa:2048</code>	Crea una nueva clave privada RSA de 2048 bits junto con el certificado.
<code>-keyout key.pem</code>	Guarda la clave privada generada en el archivo <code>key.pem</code> .
<code>-out cert.pem</code>	Guarda el certificado X.509 generado en el archivo <code>cert.pem</code> .
<code>-days 365</code>	El certificado será válido durante 365 días.

openssl

- Permite activar HTTPS / WSS
- Cifrar comunicaciones entre dispositivos
- Probar servicios sin tener que tener certificados oficiales
- Al comando se le puede añadir un parámetro para evitar que nos pida la información
- -subj “...”

openssl

Campo	Descripción
C	País (Country)
ST	Estado o provincia (State)
L	Localidad o ciudad (Locality)
O	Organización (Organization)
OU	Unidad organizativa (Organizational Unit)
CN	Nombre común (Common Name)
emailAddress	Correo electrónico

- Cada inicial lleva una barra / delante del campo.
- En el comando si añadimos **-nodes** no encripta la clave privada **key.pem**

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes \  
-subj "/C=ES/ST=Madrid/L=Madrid/O=AntonioTech/OU=IoT/CN=raspberry.local/emailAddress=antonio@ex.com"
```

Servidor WSS

- **El primer paso es configurar el contexto para SSL.**
- Tenemos que indicar los dos ficheros generados anteriormente.

```
boost::asio::ssl::context ctx(boost::asio::ssl::context::tlsv12);
```

```
ctx.set_options(  
    boost::asio::ssl::context::default_workarounds |  
    boost::asio::ssl::context::no_sslv2 |  
    boost::asio::ssl::context::no_sslv3 |  
    boost::asio::ssl::context::single_dh_use  
);
```

```
ctx.use_certificate_file("cert.pem", boost::asio::ssl::context::pem);  
ctx.use_private_key_file("key.pem", boost::asio::ssl::context::pem);
```

Diferencias entre: `boost::asio::io_context` / `boost::asio::ssl::context`

- **`boost::asio::io_context`**

- El motor principal de I/O (asíncronas)
- Coordinar eventos en conexiones TCP, temporizadores y lectura / escritura de sockets.
- Ejecutar los handlers cuando ocurren eventos

- **`boost::asio::ssl::context`**

- Contexto de configuración para TLS/SSL, para cifrar comunicaciones con HTTPS / WSS
- Certificados a utilizar
- Que claves privadas cargar
- Protocolos TLS como TLS 1.2 o 1.3)
- Opciones de seguridad

```
void run_server_1_mensaje(net::io_context& ioc, ssl::context& ctx, unsigned short port) {  
    tcp::acceptor acceptor(ioc, tcp::endpoint(tcp::v4(), port));
```

Servidor WSS

```
for (;;) {  
    tcp::socket socket(ioc);  
    acceptor.accept(socket);  
  
    ssl::stream<tcp::socket> ssl_stream(std::move(socket), ctx);  
    ssl_stream.handshake(ssl::stream_base::server);  
  
    websocket::stream<ssl::stream<tcp::socket>> ws(std::move(ssl_stream));  
    ws.accept();  
  
    beast::flat_buffer buffer;  
    ws.read(buffer);  
    ws.text(ws.got_text());  
    std::cout << "Mensaje recibido: " << beast::make_printable(buffer.data()) << std::endl;  
  
    ws.write(buffer.data());  
}
```

Servidor WSS - Pasos

- **Pasos**

- **1 – Inicializar io_context, configurar SSL/TLS e indicar el puerto**

- **2 – Inicializar el servidor**

- `tcp::acceptor acceptor(ioc, tcp::endpoint(tcp::v4(), port));`
- Crea un acceptor TCP para escuchar por el puerto indicado
- Y utilizamos IPv4

- **3 – Bucle principal para aceptar clientes:**

- `for (;;) {`
- `tcp::socket socket(ioc); // Acepta conexiones`
- `acceptor.accept(socket); // y crea un socket TCP`

Servidor WSS – Pasos II

- **4 – HandShake TLS**

- **// Crea un stream SSL sobre TCP**
- `ssl::stream<tcp::socket> ssl_stream(std::move(socket), ctx);`
- **// Realiza el handshake TLS como servidor**
- `ssl_stream.handshake(ssl::stream_base::server);`

- **5 – HandShake WebSocket**

- **// Crea un stream WebSocket sobre el canal TLS**
- `websocket::stream<ssl::stream<tcp::socket>> ws(std::move(ssl_stream));`
- **// Realiza el handshake WebSocket, para completar la conexión WSS**
- `ws.accept();`

Servidor WSS – Pasos III

- **6 – lectura del mensaje**

- **// El buffer se utiliza para los datos entrantes**
- `beast::flat_buffer buffer;`
- **// Lee un mensaje del buffer.**
- `ws.read(buffer);`

- **7 – Procesamiento y eco**

- `// Configura el mensaje para tratarlo como texto no binario`
- `ws.text(ws.got_text());`
- **// Imprime el mensaje recibido en un formato legible**
- `std::cout << "Mensaje recibido: " << beast::make_printable(buffer.data()) << std::endl;`
- **// De Vuelta al cliente, hace el eco**
- `ws.write(buffer.data());`

Cliente WSS - Pasos

- Utilizaríamos: **Boost.Beast + Boost.Asio + OpenSSL**
- **Pasos:**
 - **1- Configurar el contexto SSL**
 - `boost::asio::ssl::context ctx(boost::asio::ssl::context::tlsv12);`
 - **// Para certificados autofirmados**
 - `ctx.set_verify_mode(boost::asio::ssl::verify_none);`
 - **2- Resolver y conectar**
 - `boost::asio::io_context ioc;`
 - `tcp::resolver resolver(ioc);`
 - `auto const results = resolver.resolve("localhost", "9002");`
 - `boost::asio::ssl::stream<tcp::socket> stream(ioc, ctx);`
 - `boost::asio::connect(stream.next_layer(), results);`
 - `stream.handshake(boost::asio::ssl::stream_base::client);`

Cliente WSS – Pasos II

- **Pasos:**

- **3 – Handshake WebSocket**

- `beast::websocket::stream<boost::asio::ssl::stream<tcp::socket>>`
`ws(std::move(stream));`
 - `ws.handshake("localhost", "/");`

- **4 – Enviar y recibir**

- `ws.write(boost::asio::buffer("Hola servidor"));`
 - `beast::flat_buffer buffer;`
 - `ws.read(buffer);`
 - `std::cout << "Respuesta: " << beast::make_printable(buffer.data()) << std::endl;`

CROW C++ 17

Introducción

- **Crow** es un microframework web para C++ que te permite construir **APIs RESTful**, **servidores HTTP**, y hasta **WebSockets**,
- Sintaxis muy parecida a frameworks como Flask (Python) o Express (Node.js).
 - **Header-only**: no necesitas compilar la librería, solo incluirla.
 - **Rápido**: diseñado para alto rendimiento.
 - **Multihilo**: soporta múltiples hilos para manejar peticiones concurrentes.
 - **JSON integrado**: sin necesidad de librerías externas para manejar JSON.

Instalación de Crow mediante vcpkg

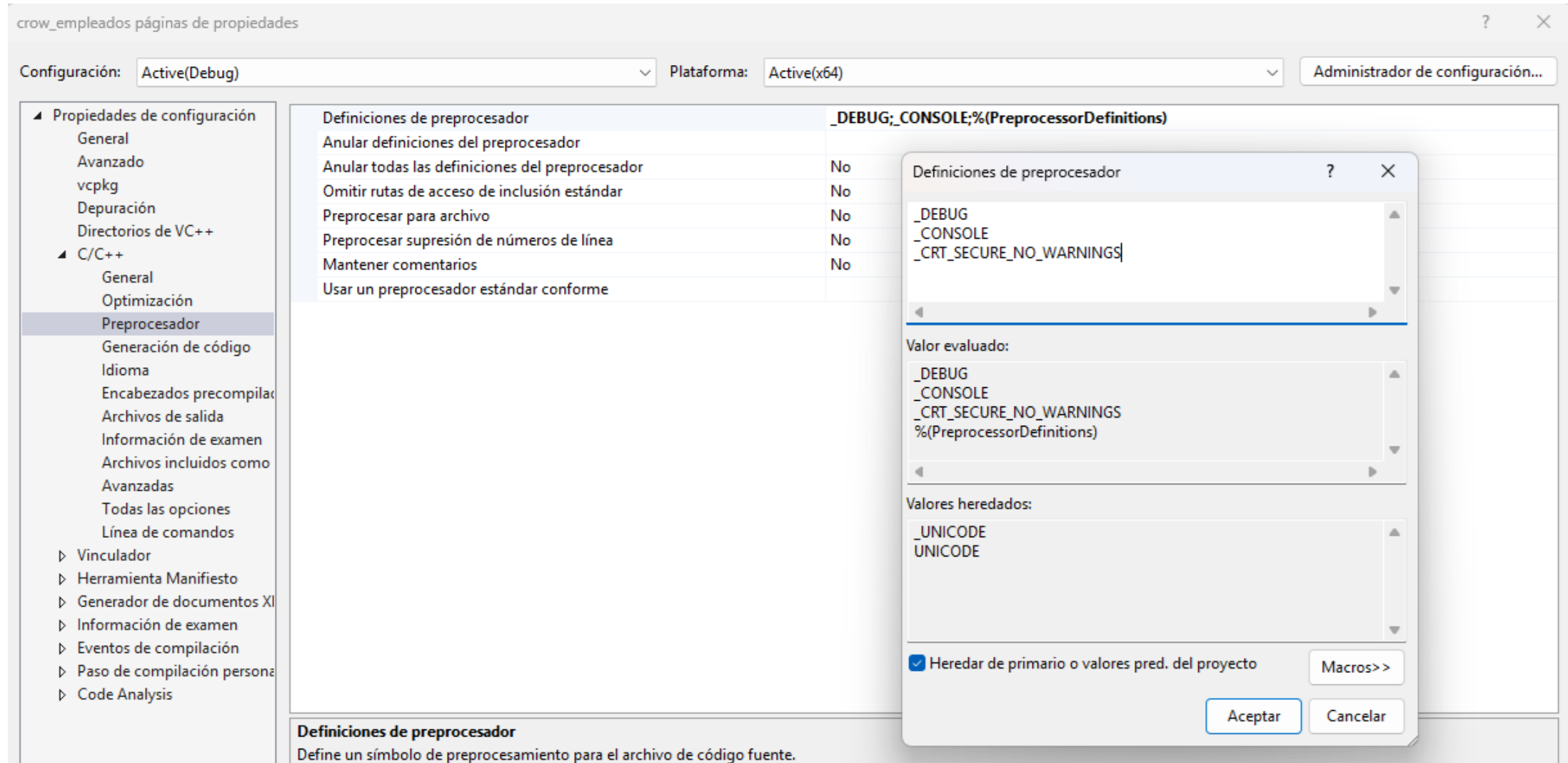
Instalación

- **vcpkg install crow**
- **vcpkg integrate install**
- Sirve para:
 - Definir operaciones CRUD
 - Servidores HTTP
 - WebSockets
 - Microservicios
 - Integración con Base de datos

Problemas con la librería crow

- A veces al compilar VS da este error:
 - Error C4996 'strcpy': This function or variable may be unsafe. Consider using strcpy_s instead. To disable deprecation, use **_CRT_SECURE_NO_WARNINGS**. See online help for details. crow_empleados C:\tools\vcpkg\installed\x64-windows\include\crow\json.h 1990
- En los proyectos se añade en:
 - Botón derecho en el Proyecto.
 - Propiedades → C/C++ → Preprocessor → Preprocessor Definitions
- En proyectos con cmake:
 - target_compile_definitions(\${PROJECT_NAME} PRIVATE _CRT_SECURE_NO_WARNINGS)

Problemas con la librería crow



Ejemplo

```
#include <iostream>
```

```
#include <crow.h>
```

```
void testCrow() {
```

```
    crow::SimpleApp app;
```

```
    CROW_ROUTE(app, "/")([]() {
```

```
        return "Hello world";
```

```
    });
```

```
    app.port(18080).run();
```

```
}
```

```
int main(){
```

```
    testCrow();
```

```
    return 0;
```

```
}
```

Respuesta en json

<http://localhost:8080/json>

```
crow::SimpleApp app;
```

```
CROW_ROUTE(app, "/json")([]() {  
    crow::json::wvalue respuesta;  
    respuesta["mensaje"] = "¡Hola desde Crow!";  
    respuesta["estado"] = "ok";  
    respuesta["codigo"] = 200;  
    return crow::response{ respuesta };  
});
```

```
app.bindaddr("127.0.0.1");  
app.port(8080);  
app.concurrency(2);  
app.run();
```

Estructura de un microservicio en C++

Estructura para producción

- **/src/main.cpp**
 - Punto de entrada del microservicio.
 - Cargas configuración
 - Inicializas Crow
 - Registras rutas
 - Arrancas el servidor
- **/src/server/**
 - Todo lo relacionado con **Crow**:
 - **crow_server.***
 - Configura el servidor HTTP
 - Define número de hilos
 - Configura middlewares
 - Arranca el servidor
- **routes.***
 - Define endpoints REST
 - Conecta con servicios de dominio
- **websocket_handlers.***
 - Handlers para WebSockets
 - Gestión de sesiones
 - Broadcast de mensajes
- **middleware/**
 - Autenticación
 - Logging
 - CORS
 - Rate limiting

Estructura para producción

- **/src/domain/**
 - **Lógica de negocio**, totalmente independiente de Crow.
 - **models/**
 - Objetos del dominio.
 - **services/**
 - Reglas de negocio.
 - **dto/**
 - Conversión JSON ↔ C++.
- **/src/infrastructure/**
 - Código de soporte:
 - Configuración
 - Logging
 - Utilidades
 - Carga de variables de entorno
- **/src/adapters/**
 - Traducción entre transporte y dominio.
 - REST ↔ dominio
 - WebSockets ↔ dominio
 - gRPC ↔ dominio
- **/tests/**
 - Tests unitarios con GoogleTest o Catch2.
 - Visual Studio detecta automáticamente los targets de test.

Estructura en producción

- En tu **CMakeLists.txt**, simplemente incluyes los archivos:

```
add_executable(mi_servicio
    src/main.cpp
    src/server/crow_server.cpp
    src/server/routes.cpp
    src/server/websocket_handlers.cpp
    src/domain/services/user_service.cpp
    src/infrastructure/logger.cpp
    src/adapters/rest_adapter.cpp
)
```

```
/MiMicroservicio/
├── CMakeLists.txt
├── CMakePresets.json
├── vcpkg.json
├── README.md
├── /src/
│   ├── main.cpp
│   ├── server/
│   │   ├── crow_server.h
│   │   ├── crow_server.cpp
│   │   ├── routes.h
│   │   ├── routes.cpp
│   │   ├── websocket_handlers.h
│   │   ├── websocket_handlers.cpp
│   │   └── middleware/
│   │       ├── auth_middleware.h
│   │       └── logging_middleware.h
│   ├── domain/
│   │   ├── models/
│   │   │   ├── user.h
│   │   │   └── order.h
│   │   ├── services/
│   │   │   ├── user_service.h
│   │   │   └── order_service.h
│   │   └── dto/
│   │       ├── user_dto.h
│   │       └── order_dto.h
│   ├── infrastructure/
│   │   ├── config.h
│   │   ├── logger.h
│   │   ├── logger.cpp
│   │   └── utils.h
│   └── adapters/
│       ├── rest_adapter.cpp
│       ├── websocket_adapter.cpp
│       └── grpc_adapter.cpp    (si usas gRPC)
└── /tests/
    ├── CMakeLists.txt
    └── test_user_service.cpp
```

Rutas, controladores y manejo de peticiones **GET/POST/PUT/DELETE**

Operaciones CRUD

- Create → POST → **crow::HTTPMethod::POST | "POST"_method**
 - Subir un recurso en el servidor.
 - El recurso se envía en el body
- Read → GET → **crow::HTTPMethod::GET | "GET"_method**
 - Recuperar un recurso del servidor
 - Normalmente se enviará un parámetro: id en la URL
- Update → PUT → **crow::HTTPMethod::PUT | "PUT"_method**
 - Actualizar un recurso del servidor
 - El recurso se envía en el body
- Delete → DELETE → **crow::HTTPMethod::Delete | "Delete"_method**
 - Eliminar un recurso del servidor
 - Normalmente se enviará un parámetro: id en la URL

Parámetros en las peticiones

- Necesitamos un objeto: **crow::SimpleApp app;**
- Sobre este se van definiendo las rutas y los métodos:
- **CROW_ROUTE(app, /ruta).methods(método)(lambda) {}**

```
CROW_ROUTE(app, "/ruta")
    .methods("GET"_method, "POST"_method)
// Opcional: especifica métodos
([](const crow::request& req){
    // Aquí va tu lógica
    return crow::response("Hola mundo");
});
```

app: instancia de `crow::SimpleApp`

"/ruta": URL que responde

req: objeto que contiene headers, cuerpo, método, etc.

- Representa el objeto **request**

Formato de las lambdas

- Sin parámetros:

```
CROW_ROUTE(app, "/ping")  
([](){  
    return "pong";  
});
```

- Con acceso al cuerpo de la petición:

```
CROW_ROUTE(app, "/echo").methods("POST"_method)  
([](const crow::request& req){  
    return crow::response(req.body);  
});
```

Con parámetros en la URL

- CROW_ROUTE(app, "/saludo/<**string**>")
- ([](std::string nombre){
- return "Hola " + nombre;
- });

Tipo de los parámetros en las peticiones

Tipo	Ejemplo de ruta	Conversión automática
<code>int</code>	<code>/suma/<int>/<int></code>	Sí
<code>uint64_t</code>	<code>/id/<uint64_t></code>	Sí
<code>double</code>	<code>/precio/<double></code>	Sí
<code>float</code>	<code>/temperatura/<float></code>	Sí
<code>std::string</code>	<code>/saludo/<string></code>	Sí
<code>char</code>	<code>/letra/<char></code>	Sí
<code>bool</code>	<code>/activo/<bool></code>	Sí (<code>true</code> / <code>false</code>)

`/productos/<int>/<double>/<string>`

Tipos de parámetros en las peticiones

- Si el tipo no se puede convertir desde la URL, Crow devuelve automáticamente un error 404.
- Los parámetros se extraen en el orden en que aparecen en la ruta.
- No se puede usar tipos complejos directamente (como `std::vector`, `struct`, etc.) en rutas, pero sí dentro del cuerpo JSON de la petición.
- Los parámetros definidos, tendrán que ser luego parámetros en las funciones lambda.

Asociación rutas → lambdas

Tipo	Ejemplo de ruta	Conversión automática
<code>int</code>	<code>/suma/<int>/<int></code>	Sí
<code>uint64_t</code>	<code>/id/<uint64_t></code>	Sí
<code>double</code>	<code>/precio/<double></code>	Sí
<code>float</code>	<code>/temperatura/<float></code>	Sí
<code>std::string</code>	<code>/saludo/<string></code>	Sí
<code>char</code>	<code>/letra/<char></code>	Sí
<code>bool</code>	<code>/activo/<bool></code>	Sí (<code>true</code> / <code>false</code>)

Si la lambda tiene que recibir la request tiene que ser el primer parámetro

Con JSON

```
CROW_ROUTE(app, "/json").methods("POST"_method)
([])(const crow::request& req){
    auto datos = crow::json::load(req.body);
    if (!datos)
        return crow::response(400);

    std::string nombre = datos["nombre"].s(); // A string
    return crow::response("Hola " + nombre);
};
```

```
// Recoger del cuerpo de la peticion el json (usuario)
auto body = crow::json::load(req.body);
if (!body) {
    return crow::response(400, "Json incorrecto");
}
```

```
// Dar de alta el usuario en la coleccion:
std::lock_guard<std::mutex> lock(mtx);
int id = this->siguiente_id++;
usuarios[id] = std::move(body);
```

Ojo, cuando queremos guardar el json recibido en Post o Put hay que hacer move, si no, lo veremos mal Codificado.

Con JSON

- Los datos que vienen por la request en json se cargan con:
 - `auto datos = crow::json::load(req.body);`
 - Devuelve un objeto de tipo: **`crow::json::rvalue`**
- Si los datos enviados son:

```
{  
  "nombre": "Ana",  
  "edad": 42  
}
```
- Para utilizarlos en C++:
 - `std::string nombre = datos["nombre"].s();` `// Extrae "Ana"`
 - `int edad = datos["edad"].i();` `// Extrae 42 como entero`

Otros métodos relacionados

Método	Tipo JSON esperado	C++ equivalente
<code>.s()</code>	string	<code>std::string</code>
<code>.i()</code>	integer	<code>int</code>
<code>.d()</code>	decimal/float	<code>double</code>
<code>.b()</code>	boolean	<code>bool</code>
<code>.t()</code>	timestamp	<code>time_t</code>

Tipos relacionados con JSON en crow

Tipo	¿Qué es?	¿Cuándo se usa?
<code>crow::json::rvalue</code>	Representa un valor JSON leído (resultado de <code>json::load</code>)	Para acceder a campos del JSON recibido
<code>crow::json::wvalue</code>	Representa un valor JSON que vas a construir o devolver	Para crear respuestas JSON
<code>crow::json::json_return_type</code>	Alias para <code>crow::json::wvalue</code>	Se usa como tipo de retorno en rutas que devuelven JSON

Ejemplos

- **rvalue** para **extraer** datos del JSON
 - `auto datos = crow::json::load(req.body);`
 - `std::string nombre = datos["nombre"].s();`
- **wvalue** para **construir** un JSON para devolverlo como respuesta
 - `crow::json::wvalue respuesta;`
 - `respuesta["estado"] = "ok";`
 - `respuesta["codigo"] = 200;`
 - `return crow::response(respuesta);`

Acceso a headers, query params

```
CROW_ROUTE(app, "/info")
([])(const crow::request& req){
    std::string agente = req.get_header_value("User-Agent");
    std::string param = req.url_params.get("id"); // /info?id=123
    return crow::response("Agente: " + agente + ", ID: " + param);
});
```

Puesta en marcha del Servidor

```
#include <crow.h>
```

```
int main() {
```

```
    crow::SimpleApp app;
```

```
    CROW_ROUTE(app, "/")
```

```
    ([](){
```

```
        return "¡Servidor Crow en marcha!";
```

```
});
```

```
    app.port(18080)
```

```
    // Puerto donde escucha el servidor
```

```
    .concurrency(4)
```

```
    // Número de hilos (multithreading)
```

```
    .multithreaded()
```

```
    // Activa el modo multihilo
```

```
    .run();
```

```
    // Arranca el servidor
```

```
}
```

Puesta en marcha Servidor

- **app.port(<número>)**
 - Define el **puerto TCP** donde el servidor escucha.
 - Ejemplo: app.port(8080) → accesible en <http://localhost:8080>
- **app.concurrency(<número>)**
 - Número de **hilos de ejecución** que Crow usará para manejar peticiones simultáneas.
 - **Valor recomendado: igual al número de núcleos del procesador.**
 - Ejemplo: app.concurrency(4) en una Raspberry Pi de 4 núcleos.
- **app.multithreaded()**
 - Activa el **modo multihilo**.
 - Si no lo usas, Crow funcionará en modo monohilo (una petición a la vez).
- **app.run()**
 - Lanza el servidor y lo deja escuchando indefinidamente.

Consideraciones

- Si no usamos **multithreaded()**
 - **crow** seguirá funcionando, pero solo podrá atender **una petición a la vez**, lo cual no es ideal si esperas concurrencia o múltiples clientes.

```
app.port(8080)
  .concurrency(std::thread::hardware_concurrency())
  .multithreaded()
  .run();
```

Gestión de JSON

Integración con nlohmann/json

Parsear datos en Json

- Librería: **nlohmann-json**
- **vcpkg install nlohmann-json**
- **vcpkg integrate install**
- En el proyecto hacemos referencia a:
- **#include <nlohmann/json.hpp>**

```
#include <iostream>
#include <sstream>
#include <nlohmann/json.hpp>

void testJson() {
    nlohmann::json doc;

    doc["curso"] = "Microservicios en C++";
    doc["horas"] = 25;
    doc["tecnologias"] = { "xml", "json", "rest", "soap" };
    std::cout << "Json: " << doc.dump(4) << std::endl;
    std::cout << "curso: " << doc["curso"] << std::endl;
}
```

```
void strToJson() {  
    // Se define una cadena Raw: con formato json  
    std::istringstream ss(R"({"nombre":"Ana","edad":28,  
        "intereses":["programacion","musica","senderismo"]}");  
    nlohmann::json doc;  
  
    // Se convierte a json:  
    ss >> doc;  
    std::cout << "nombre: " << doc["nombre"] << std::endl;  
    std::cout << doc << std::endl;  
}
```

Vector / Grabar a fichero

- Se pueden definir vectores del tipo: **nlohmann::json** para almacenar objetos json:
 - `std::vector<nlohmann::json> array;`
 - `nlohmann::json grupo;`
 - `grupo = array; // Se convierte automáticamente`
 - **`grupo.dump(4)`** // Añade indentación, el resultado se puede grabar en un fichero o se imprime por la pantalla.
- Utilizar operador `<<`, con un objeto **`std::ofstream`** o **`std::cout`**

Objetos a Json

```
nlohmann::json Pedido::to_json() const
{
    return nlohmann::json{ {"idpedido", this->idpedido},
        {"cliente", this->cliente},
        {"empresa", this->empresa},
        {"empleado", this->empleado},
        {"importe", this->importe},
        {"pais", this->pais}};
}
```

De JSON a Objeto

```
Pedido Pedido::from_json(const nlohmann::json& j){  
    Pedido p;  
  
    p.idpedido = j.at("idpedido").get<std::string>();  
    p.cliente = j.at("cliente").get<std::string>();  
    p.empresa = j.at("empresa").get<std::string>();  
    p.empleado = j.at("empleado").get<std::string>();  
    p.importe = j.at("importe").get<float>();  
    p.pais = j.at("pais").get<std::string>();  
  
    return p;  
}
```

Validaciones de datos

```
bool validar_json(const nlohmann::json& j) {  
    return j.contains("idpedido") && j["idpedido"].is_number_integer() &&  
        j.contains("cliente") && j["cliente"].is_string() &&  
        j.contains("empresa") && j["empresa"].is_string() &&  
        j.contains("empleado") && j["empleado"].is_string() &&  
        j.contains("importe") && j["importe"].is_number() &&  
        j.contains("pais") && j["pais"].is_string();  
}
```

Manejar errores al parsear json

```
try {  
    nlohmann::json j = nlohmann::json::parse(req.body);  
    Pedido pedido = Pedido::from_json(j);  
  
    // procesar pedido...  
} catch (const std::exception& e) {  
    return crow::response(400, std::string("Error al procesar JSON: ") + e.what());  
}
```

crow y nlohmann

- La librería **crow** utiliza por debajo **nlohmann**, pero de una forma limitada.
- Al utilizar en un método **post**
 - `nlohmann::json j = nlohmann::json::parse(req.body);`
- En vez:
 - `auto body = crow::json::load(req.body);`
- Tenemos mas control, para empezar con los errores.
- Si viene mal el json, con crow obtendríamos un objeto vacío, pero nlohmann.
- Se integra con colecciones, mejor para estructuras anidadas

Integración con clases C++

- `crow::json::rvalue` no tiene soporte directo para convertir a clases C++.
- **`nlohmann::json`** permite usar **`from_json()`** y **`to_json()`** para convertir entre JSON y objetos C++ fácilmente.
- **Cuando usar `crow::json::load`** → cuando tengamos una lógica sencilla y directa.
- **`nlohmann::json::parse()`** si necesitas **validación robusta, estructuras anidadas, o convertir JSON a clases C++** como Pedido
 - Permite que definamos funciones **`to_json()`** y **`from_json()`**

Integración con clases C++

- Hay que definirlas fuera de la clase.
- Nos a permite hacer cosas como estas:
 - `nlohmann::json j = pedido;      // usa to_json`
 - `Pedido p = j.get<Pedido>();      // usa from_json`
- Según entra la petición (en un método post), podemos hacer:
 - `nlohmann::json j = nlohmann::json::parse(req.body);`
 - `Pedido p = j.get<Pedido>()`

Integración con clases C++

```
#include <nlohmann/json.hpp>
using json = nlohmann::json;
```

```
class Pedido {
public:
    // Definición de atributos
};
```

// Fuera de la clase, en el mismo namespace, definir las funciones:

```
void to_json(json& j, const Pedido& p){
    j = json{{"att1", p.att1}, {...} ... };
}

void from_json(const json& j, Pedido &p){
    j.at("idpedido").get_to(p.idpedido);
    j.at("cliente").get_to(p.cliente);
    ...
}
```

Manejo de errores y buenas prácticas en respuesta HTTP

Manejo de códigos de error y optimización del flujo de datos

- Crow permite devolver respuestas con códigos personalizados.
 - Para ello aplicamos el método response con el código HTTP y un mensaje:
 - `return crow::response(400, "Solicitud mal formada");`
 - `return crow::response(404, "Recurso no encontrado");`
 - `return crow::response(500, "Error interno del servidor");`
- **Códigos de respuesta HTTP:**
 - <https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status>

Manejo de códigos de error y optimización del flujo de datos

- Tenemos la opción de devolver un json:

```
nlohmann::json error = {  
    {"codigo", 400},  
    {"mensaje", "Falta el campo cliente"}  
}  
return crow::response(400, error.dump());
```

Logging básico del servicio

Implementación de registros de errores y mensajes de diagnóstico

- **Crow** no tiene un sistema de **logging avanzado**, pero podemos utilizar unas macros ya definidas:
 - **CROW_LOG_***
- **Macros definidas en crow** para emitir logs personalizados:
 - CROW_LOG_DEBUG << "Mensaje de depuración";
 - CROW_LOG_INFO << "Servidor iniciado correctamente";
 - CROW_LOG_WARNING << "Parámetro inesperado recibido";
 - CROW_LOG_ERROR << "Error al procesar la petición";
 - CROW_LOG_CRITICAL << "Fallo grave en el sistema";
- Para proyectos más grandes podemos integrar crow con otras **librerías externas**:
 - **splog, Boost.Log, log4cpp**

Niveles de log

Nivel	Descripción
Debug	Máxima información (ideal para desarrollo)
Info	Eventos importantes
Warning	Cosas que podrían causar problemas
Error	Fallos que afectan el funcionamiento
Critical	Errores graves que requieren atención inmediata

Activación del sistema de logs

- Se puede activar de dos formas:
 - **Cuando estamos configurando y arrancando el servidor:**

```
app.port(8080)
  .concurrency(std::thread::hardware_concurrency())
  .multithreaded()
  .loglevel(crow::LogLevel::Debug)
  .run();
```
 - **Antes de ejecutar la app, configurar el nivel de log:**
 - `crow::logger::setLogLevel(crow::LogLevel::Debug);`
- Se puede activar los logs con un nivel de log, pero son automáticos.
- El **nivel** de **log** por defecto es **info**.

Ejemplo

```
CROW_ROUTE(app, "/pedido").methods(crow::HTTPMethod::POST)([](const crow::request& req){

    try {
        nlohmann::json j = nlohmann::json::parse(req.body);
        Pedido p = j.get<Pedido>(); // Serializa el json a un objeto Pedido
        CROW_LOG_INFO << "Pedido recibido: " << p.idpedido;
        return crow::response(201, "Pedido creado");

    } catch (const std::exception& e) {
        CROW_LOG_ERROR << "Error al procesar pedido: " << e.what();
        return crow::response(400, std::string("Error: ") + e.what());
    }

});
```

Estrategias para optimizar el flujo de datos

- **Validación temprana y estructurada:**
 - Validar el objeto json al principio del método
- **Deserialización eficiente:**
 - Apoyarnos en las funciones que proporciona `nhlmann` para serializar / deserializar objetos.
- **Evitar copias innecesarias:**
 - En las funciones siempre utilizar referencias, evitar paso de objetos por valor y si no, necesitamos el objeto podemos utilizar la función `std::move`
- **Respuestas compactas y precisas**
 - Sólo devolver los datos que necesitamos, para evitar sobrecargas.
- **Multihilo y concurrencia**
 - `app.port(8080).multithreaded().run();`
- **Uso de caché:**
 - Para datos que cambian poco se pueden utilizar colecciones para mantener en memoria, si no, utilizar bases de datos clave-valor como Redis para mantener datos en una caché.

APÉNDICE

*Comparación y selección de
herramientas según el tipo de
aplicación*

Cuando utilizar una u otra tecnología

- **WebSocket** ideal cuando necesitamos:
 - Comunicación bidireccional en tiempo real.
 - Actualizaciones instantáneas sin tener que hacer **polling** constante.
 - Menor sobrecarga que HTTP con conexiones persistentes.

boost.beast vs crow

Característica	Crow	Boost.Beast
 Nivel de abstracción	Alto (microframework estilo Flask/Express)	Bajo (control total sobre HTTP y sockets)
 Multihilo	Sí, integrado	No directamente, debes usar Boost.Asio
 JSON integrado	Sí (con su propio sistema)	No, debes usar una librería externa
 Routing	Sí, muy sencillo (<code>CROW_ROUTE</code>)	No, lo implementas tú manualmente
 RESTful API	Ideal para montar APIs rápidamente	Requiere más trabajo para estructurar rutas
 Control de bajo nivel	Limitado (Crow abstrae mucho)	Total (manejas buffers, headers, etc.)
 WebSockets	Soportado	Soportado (más flexible)
 Documentación	Clara y sencilla	Técnica y extensa
 Dependencias	Ligera (header-only)	Requiere Boost completo
 Testing y depuración	Fácil de probar con mocks	Más complejo, pero más preciso
 Aprendizaje	Rápido para principiantes	Requiere experiencia con Boost.Asio

Elegir crow cuando

- Quieres montar una API RESTful rápida y sencilla.
- Prefieres una sintaxis clara y moderna.
- No necesitas control de bajo nivel sobre sockets o buffers.
- Estás trabajando en un microservicio o backend ligero.
- Quieres algo que funcione bien en Windows y Linux con vcpkg.

Elegir Boost.Beast

- Necesitas control total sobre el protocolo HTTP.
- Estás construyendo un servidor de alto rendimiento o personalizado.
- Quieres integrar con otros componentes de Boost (Asio, SSL, etc.).
- Estás trabajando en aplicaciones que requieren WebSockets avanzados.
- Buscas máxima flexibilidad y rendimiento.

Ejemplo

- Para un CRUD de una entidad de la BD con rutas del estilo: /empleados/<id> y respuesta en JSON: **crow**
- Para un servidor que maneja miles de conexiones simultaneas, con control de headers, trailers y buffers: **Boost.beast**
 - Los **trailers**: se refieren a **campos de encabezado que se envían después del cuerpo del mensaje**, en lugar de antes como los headers tradicionales.
 - Se usan cuando ciertos valores no están disponibles al inicio de la transmisión, como:
 - **Checksum o hash** del cuerpo (por ejemplo, Content-MD5)
 - **Firma digital**